UNSUPERVISED LEARNING: we do not have the samples

let's represent the MINST digits dataset test:

$f: X \to Y$

X: pictures of digits

Y: $\{0, 1, \dots, 9\}$

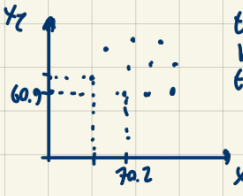
$S = (img_1, f(img_1)), (img_2, f(img_2)), \dots, (img_{1000}, f(img_{1000}))$

S: TRAINING SET

In UNSUPERVISED LEARNING, we will NOT HAVE SAMPLES ANYMORE. We will analyze the problem of CLUSTERING, where instances are all grouped for similarity.

CLUSTERING: given a set, partion the set in different groups

- GROUP SIMILAR OBJECTS (points)
- SEPARATE DISSIMILAR OBJECTS



this is just a bunch of data. We don't have classes. So we have to group similar objects

CAN BE CONTRADICTORY

There is NO UNIQUE FORMALIZATION for CLUSTERING (we will see an example that tries, failing)

* Consider an example with many similar points: (similarity measure like euclidean distance)

$x_1, x_2, \dots, x_m$; x_i is similar to x_{i+1} $\forall i$

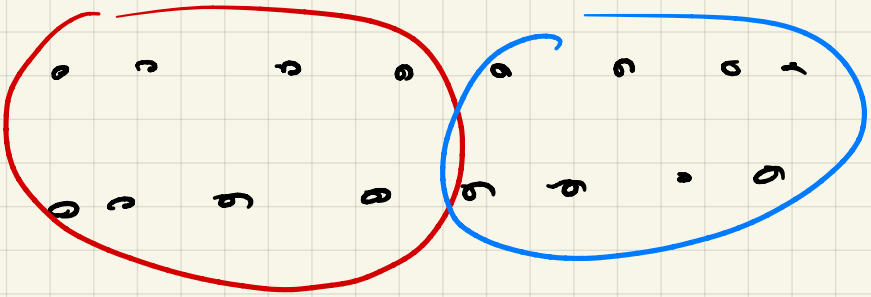
So, subsequent points are similar, but distant are not. But we said that similarities must be grouped. So we can't group x_1, x_m even though they respect similarity.

In clustering, there are a few possible approaches.

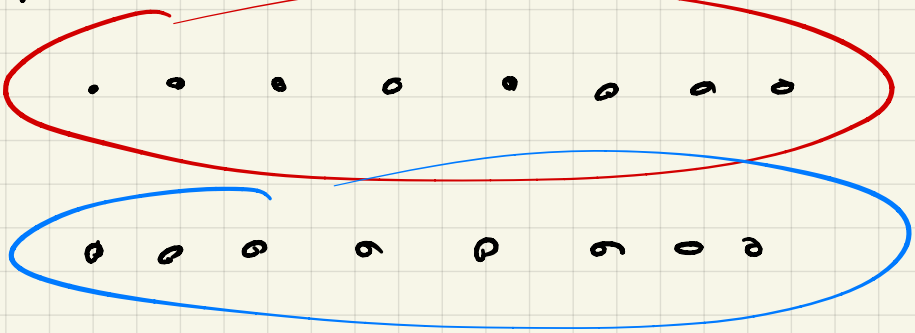
EXAMPLE

Given a set of points, I want to not separate closed-by points:

1ST APPROACH



2ND APPROACH



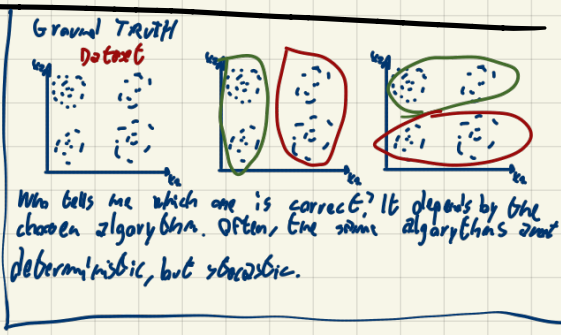
Both are correct!

THERE IS NOT A SINGLE CORRECT APPROACH. We don't have a "Ground Truth"

FORMAL MODEL FOR CLUSTERING

INPUT:

- Instance space: $X \subseteq X'$ (simply the domain of interest)



- Distance function: $d: X \times X \to \mathbb{R}_+$ (a function that gives us how distant the points are)
  - $d(x,x) = 0 \quad \forall x \in X$ // naive
  - $d(x,y) = d(y,x), \ \forall x,y \in X$ // symmetric property
  - TRIANGLE INEQUALITY // required to implement any distant function, like the other properties

$$d(x,y) + d(y,z) \geq d(x,z)$$

OUTPUT:

FOR SOME ALGORITHMS, THERE ARE MORE POSSIBLE CORRECT CLUSTERINGS

- $C = (C_1, \dots, C_k)$ (clusters set)
  - PARTITION SATISFACTION: $\bigcup_{i=1}^{k} C = X$

(OR DENDOGRAM) (hierarchy based cluster) → TREE-BASED REPRESENTATION OF CLUSTERS

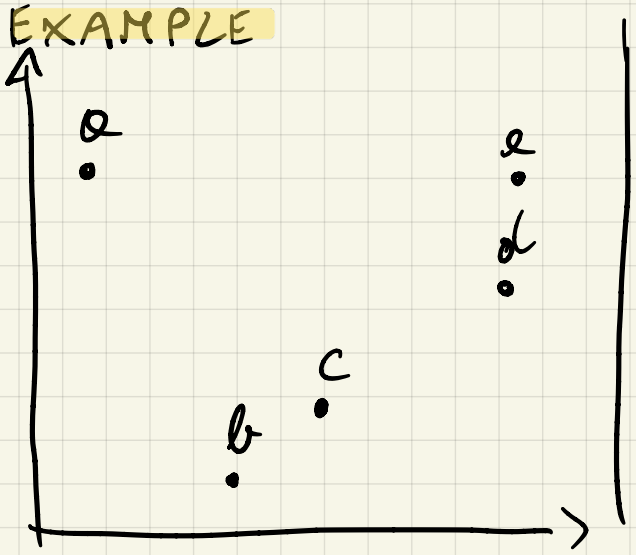
$C_i \cap C_j = \emptyset$ for $i \neq j$

A First example of algorithm (it is a family of algorithms)

LINKAGE-BASED CLUSTERING (outputs DENDOGRAM)

- Start with clustering $C_i = \{x_i\}$ for $x_i \in X$
  - ↳ EVERY POINT IS A CLUSTER
- Repeat until TERMINATION CONDITION: (it terminates when there is NO change to make anymore)
  - Merge "closest" clusters of current partition
- Return current partition

EXAMPLE



PARTITION NUMBER

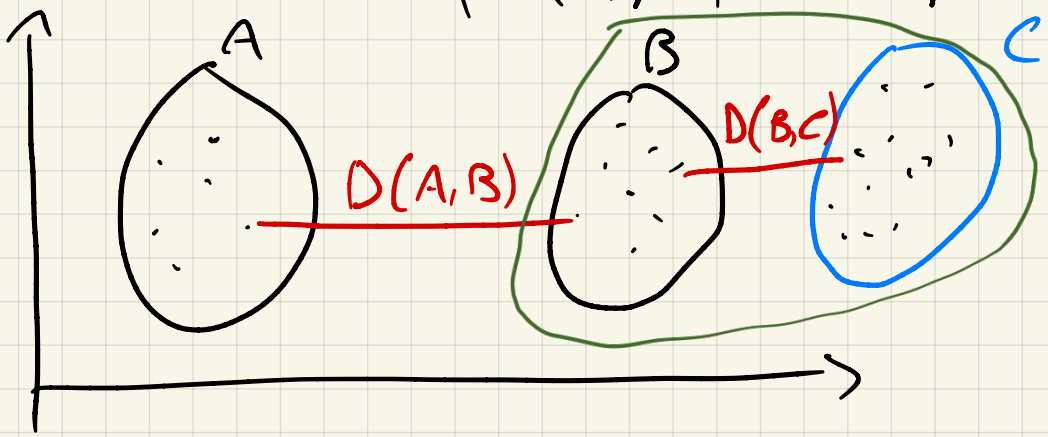
0 $\{a\}\ \{b\}\ \{c\}\ \{d\}\ \{e\}$ C

How do I proceed now?

Let's define BETWEEN-CLUSTER DISTANCE $D(A, B)$

- SINGLE LINKAGE CLUSTERING
  - $D(A, B) = \min \left(d((x, y)) \mid x \in A, y \in B \right)$ (minimum distance between the 2 clusters)

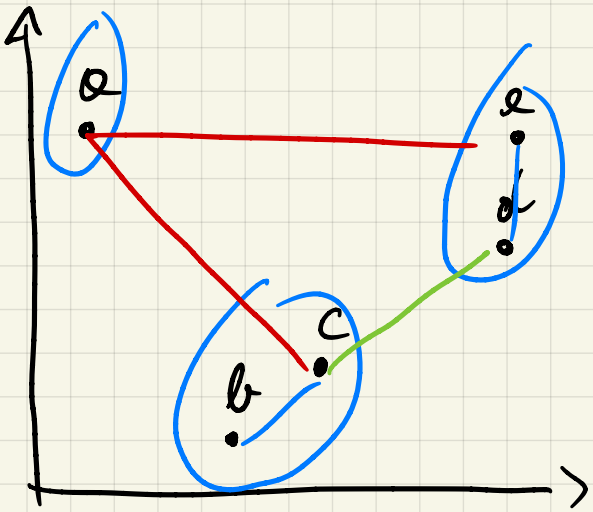
do the same for all clusters and merge the ones that are the closest

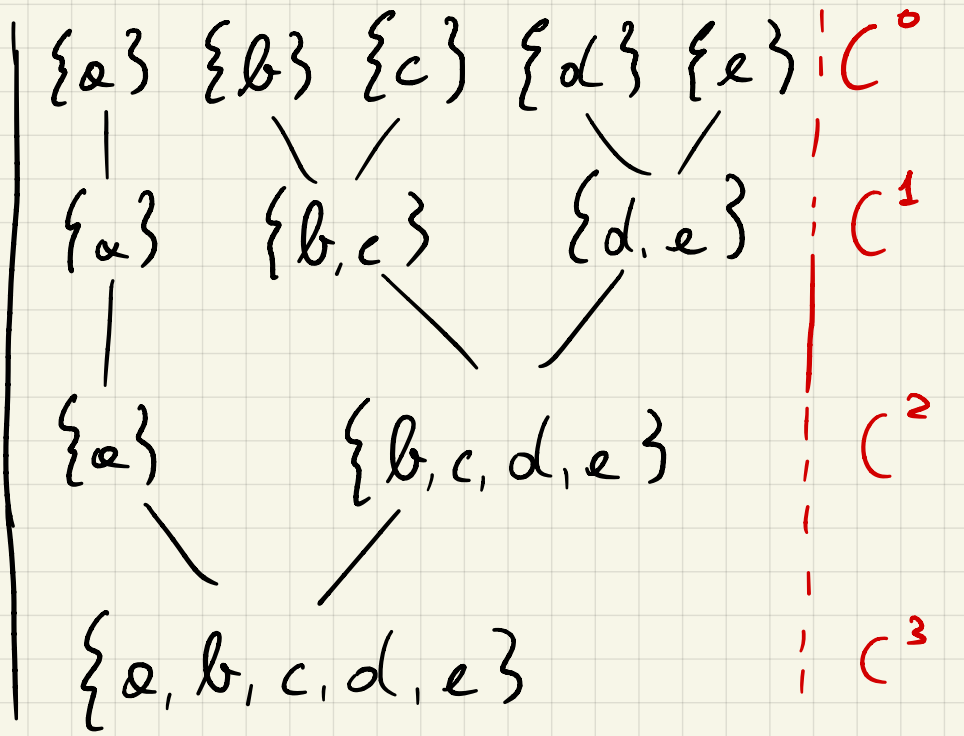


B ∪ C, in this case

  - Then merge for A and B (not the example ones, in the example we should merge B and C)

BACK TO THE EXAMPLE





If you don't consider any TERMINATION CONDITION, you will always have C^3 as final partition. However, your output will be a DENDOGRAM, so you could pick any C^i you want.

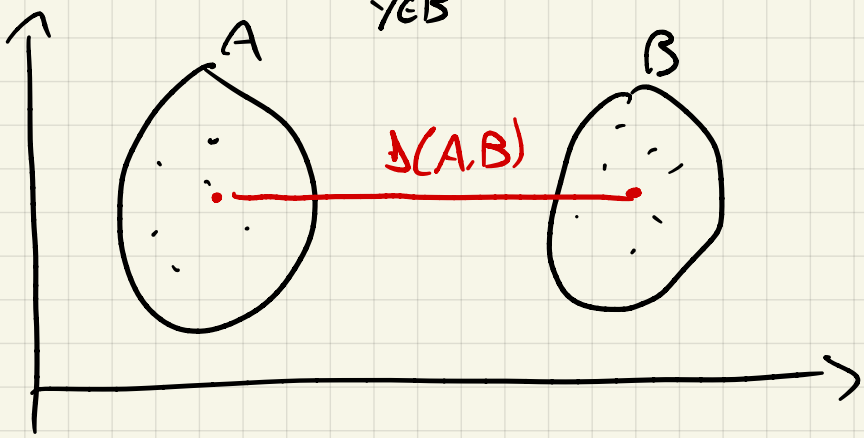
Problem! like stopping before we should optimize

This way, the algorithm is VERY EXPENSIVE (w/out any TERMINATION COND.)

- AVERAGE LINKAGE CLUSTERING

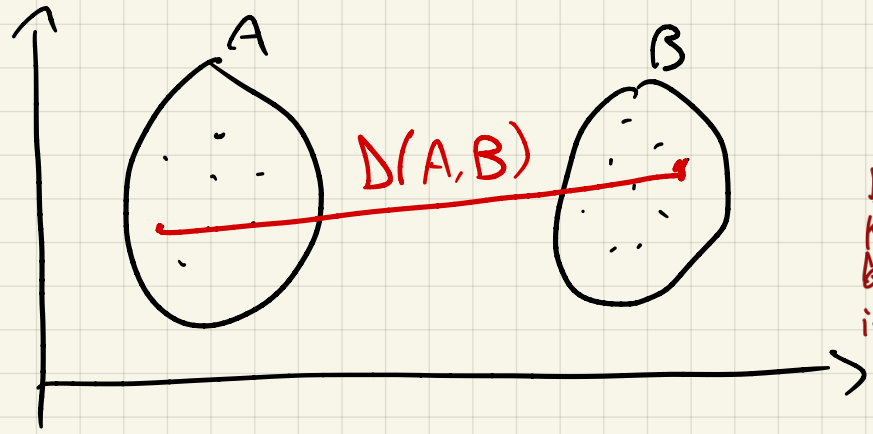
$$D(A,B) = \frac{1}{|A||B|} \sum_{\substack{x \in A \\ y \in B}} d(x,y)$$

← Changes only the distance the average distance for 2 specific clusters



- MAX LINKAGE CLUSTERING

$$D(A,B) = \max \{ d(x,y) \mid x \in A, y \in B \}$$



In general, i test methods specifics algorithms, but you test everything and decide which is the best. Even if theoretically you know which algorithms is better, sometimes test practice is different.

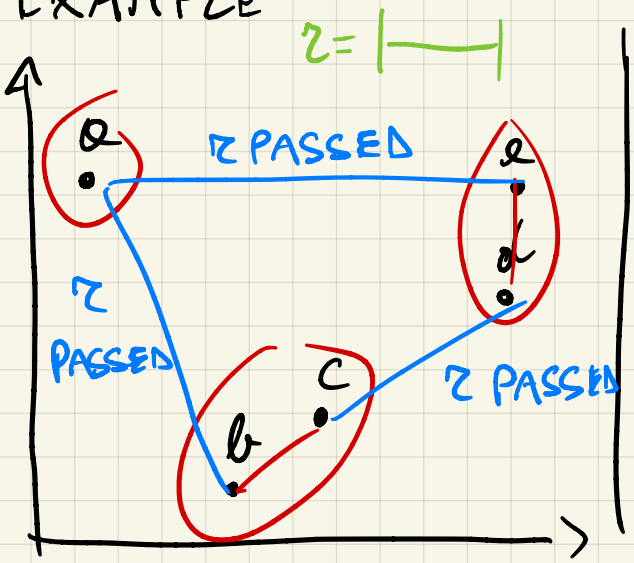
TERMINATION CONDITION (we can define always our example of termination criteria)

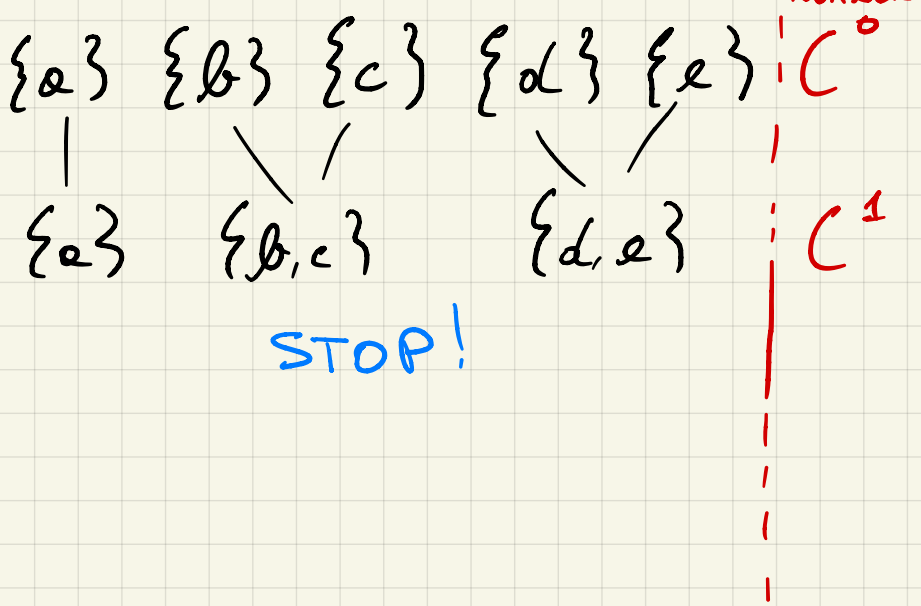
- # of clusters (provided as input)

|| there are others algorithm termination examples in the book

- DISTANCE UPPER BOUND: ($r \in \mathbb{R}$, is a max threshold of distance)
  - All between-cluster distance pass DISTANCE THRESHOLD
  - ... other bounds can be implemented

EXAMPLE





However, linkage-based clustering is EXPENSIVE in complexity

$\sim O(n^2)$, with n points

↓ in general this is a problem of clustering

Let's see other possible types of clustering.

K-MEANS CLUSTERING (you have to minimize an objective function)

- Clustering is seen as an OPTIMIZATION PROBLEM

INPUT: (X, d) $X \subseteq X'$ (distance function d on X') — for example, the euclidean distance

- # of clusters K // the goal to reach

OUTPUT:

- Partition $C = (C_1, \dots, C_k)$ (into k clusters)

G: objective function (evaluates how BAD the partition is)

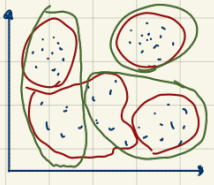
- INPUT:
  - dataset (with d) (X, d)
  - $C_1, \dots, C_k$
- OUTPUT:
  - value from $\mathbb{R}_+$

K-MEANS (and variants) searches for $C = (C_1, \dots, C_k)$ that minimizes $G((X, d), (C_1, \dots, C_k))$

Basically:

K-MEANS OPTIMIZATION PROBLEM: find $C_1, \dots, C_k$ s.t. $G((X, d), (C_1, \dots, C_k))$ is minimal

$$C = \underset{(C_1, \dots, C_k)}{\operatorname{argmin}}\ G((X, d), (C_1, \dots, C_k))$$



many ways to classify the dataset but we need the one that minimize the $G(\cdot)$

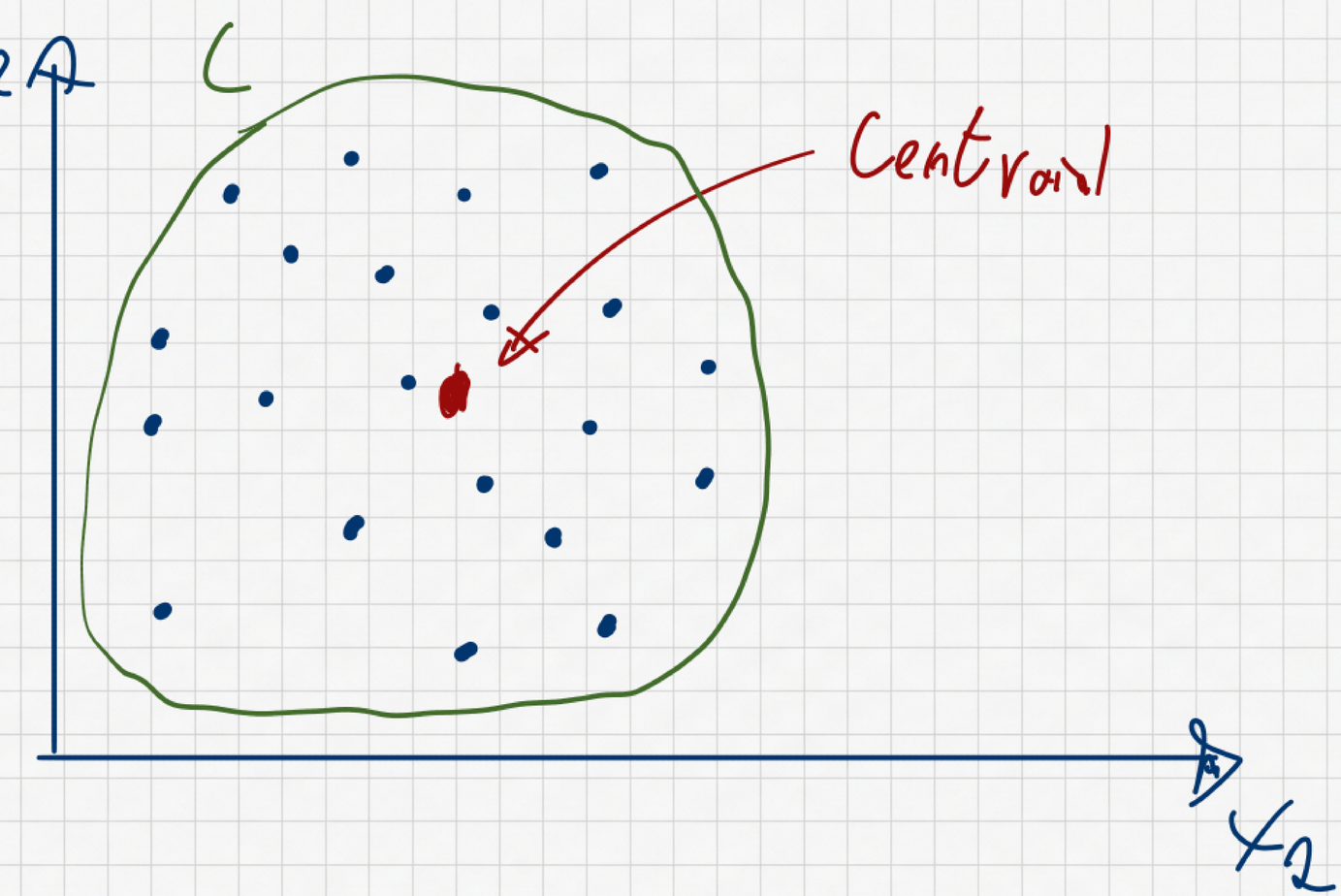
for large datasets, is computationally unfeasible

To focus on understanding the OBS. FUNCTION, we need to introduce CENTROIDS

CENTROID

For a cluster C_i, it is defined a CENTROID a function:

Centroid of Cluster $C \subseteq X \subseteq X'$:



So the Centroid is a kind of the Center of the cluster:

$$C_i, \mu_i := \operatorname{argmin} \sum_{\mu \in X'} d(x, \mu)^2$$

$$\mu_i = \underset{\mu \in X'}{\operatorname{argmin}} \sum_{x \in C_i} d(x, \mu)^2$$

It is the function that minimizes the squared distance between all points in a cluster

(if d is euclidean distance: $\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$)

Now, we can define G

K-MEANS OBJECTIVE FUNCTION G

EUCLIDEAN DISTANCE

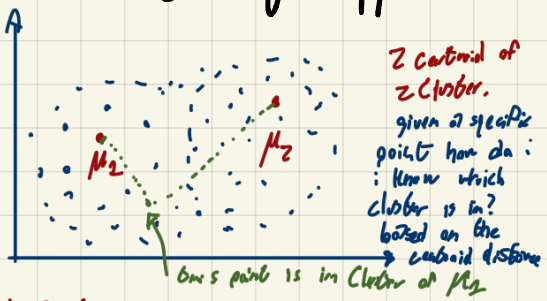
$$G_{k\text{-means}}((X, d), (C_1, \dots, C_k)) = \sum_{i=1}^{k} \sum_{x \in C_i} d(x, \mu_i(C_i))^2$$

centroid

it says that we want that the points around the centroid are compact for each cluster

This solution is very complex to find, so a lot of approximations will be used.

Let's see the algorithm model.



2 centroid of 2 cluster. given a specific point how do I know which cluster is in? based on the centroid distance

K-MEANS

INPUT: K: # of clusters; $X \subseteq \mathbb{R}^m$: feature space; d = EUCLIDEAN DIST.

m-dimensional feature space.

- INITIALIZE: Randomly choose centroids $\mu_1, \dots, \mu_K$!!!
- REPEAT UNTIL CONVERGENCE:

$$\forall i \in \{1, \dots, k\} \text{ let } C_i = \{x \in X \mid i = \underset{j}{\operatorname{argmin}} \|x - \mu_j\|\}$$

clusters

consider each point of the Dataset, if the centroid is the closest to the point. Then is part of the Cluster

$$\forall i \in \{1, \dots, k\} \quad \mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

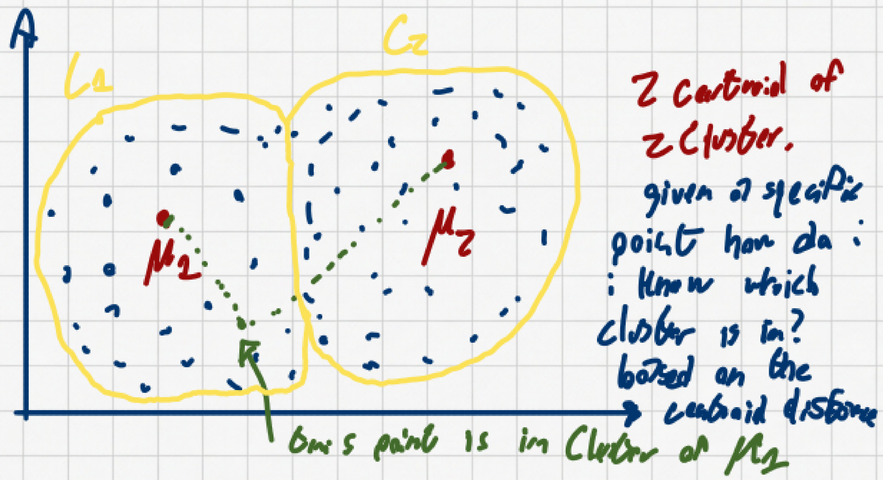
Notice: is basically a Mean/Average value → that's why is called k-means.

THESE ARE THE CENTROIDS AFTER ASSIGNING EVERY X TO A CLUSTER

A → a compute again the new centroid

So, you have the random centroid, you assign $\forall x \in X$ basing on centroids, then you compute the new centroids (based on the actual clusters), then you re-assign x to new clusters based on new centroids and so on and so forth until no changes can be made → you reach convergence

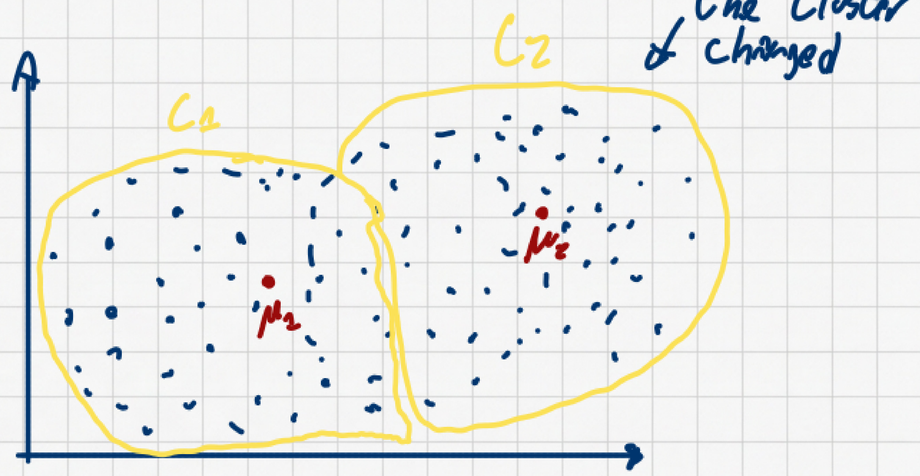
1°



2 centroid of 2 cluster. given a specific point how do : know which cluster is in? based on the centroid distance

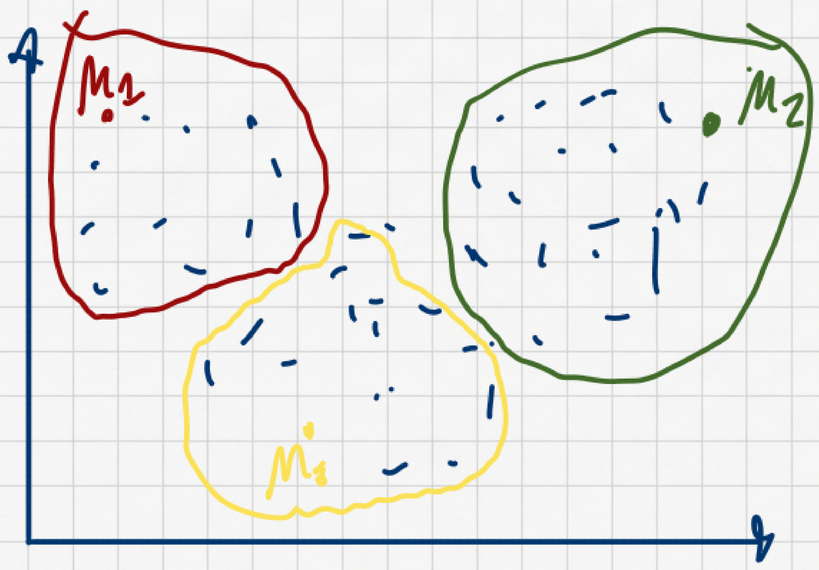
Now i add more data

in a way that also the centroid changes



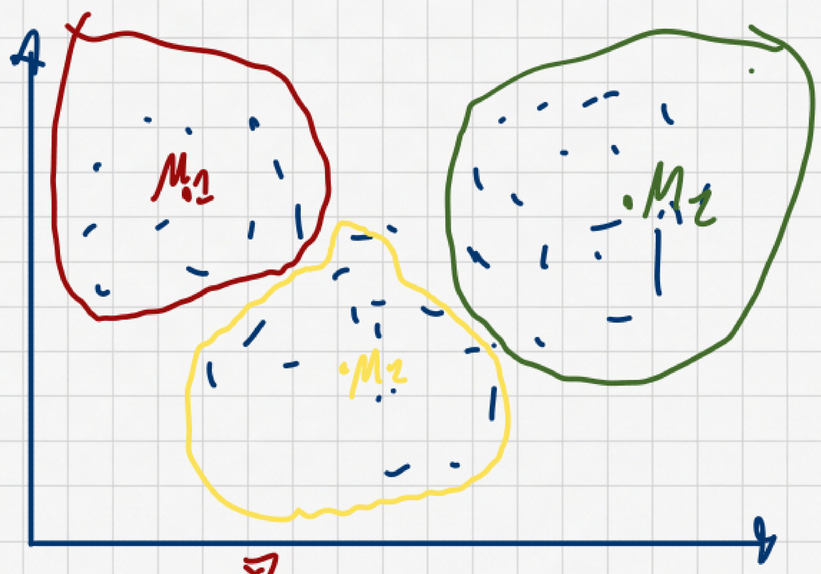
the Cluster changed

Initial step: μi randomly choosed,



compute the real centroid of those Clusters

Initial step: μi randomly choosed,



But then we have to check again if the centroid is actually the closest to each point

VARIANTS: | the main difference is the fact that the centroid has to be an actual point of the cluster-dataset, while in the k-means can be also a point $\notin X$

- k-MEDOIDS

In this way we punish more those point far more distant

$$\text{MEDOID}: \quad \mu_i(C_i) = \underset{\mu \in X}{\operatorname{argmin}} \sum_{x \in C_i} d(x,\mu)^2$$

- k-MEDIAN

$$\text{MEDIAN}: \quad \mu_i(C_i) = \underset{\mu \in X}{\operatorname{argmin}} \sum_{x \in C_i} d(x,\mu)$$

N.B.: When you compute μ_i you will pass through EVERY point in your dataset

---

LEMMA

Each iteration of k-MEANS will not increase the objective function. | this tells us: as we iterate, we find something that for sure is not worse.
Notice: the algorithm strongly depends by the initial chosen centroid: | you may get different clusters with this algorithm.

So, our algorithm is truly dependent from the initial centroids (different initial centroids lead to different clusters after convergence).

NON-GUARANTEES

- We don't know the necessary # of iterations to converge
- NO APPROXIMATION GUARANTEE: I can't be sure to be sure to find a solution that is lower then an arbitrary bound (no local minimum guarantee) → might converge to solution but it is not even a local minimum

What can we do to improve this algorithm?

(PARTIAL) SOLUTION: REPEATED EXECUTIONS ← simply test different INITIAL initialization

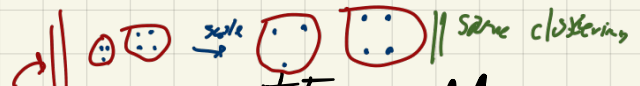
FURTHER ISSUE:
- Hyperparameter k (which is the right k?) ← how many clusters do i need??
- Test with different values of k ← just like we do in KNN

Returning to clustering, we still don't have a formalized structure for clustering. The attempts were numerous:

- KLEINBERG (2003): AXIOMATIZATION OF CLUSTERING FUNCTION

$$F((X,d)) = (C_1, \dots, C_k)$$ in general

|| (DESIRABLE) PROPERTIES OF F:



- SCALE INVARIANCE: F returns the same partition after scaling the data
- RICHNESS: $\forall C_i \in (C_1, \dots, C_k)\ \exists d$ distance function s.t. $F((X,d)) = (C_1, \dots, C_k)$ (so, distance function DETERMINES the output)
- CONSISTENCY: similar points will be clustered together and dissimilar points will be separated

We can prove an impossibility result:

IMPOSSIBILITY RESULT

So basically we don't have the guarantee that the cluster is perfect and we have an Ard a cluster which is good in respect of which we give scrolling

$\nexists$ clustering function F that satisfies at the same time SCALE INVARIANCE, RICHNESS AND CONSISTENCY.

NEURAL NETWORKS

Initially, we approached to the optimization problem parametrically by minimizing the expected risk (or loss function).

$f: X \to Y$ $\quad S = \{(x_1, t_1), \dots, (x_m, t_m)\}$ $\quad t_i = f(x_i)$

H $\quad h(x) = h(x;\vartheta) = w_1 x_1 + w_2 x_2 + w_0 \quad \vartheta = \{w_1, w_2, w_0\}$

$L(h_S)$ is the loss function or ERM